HARDEN A CONTAINER IMAGE

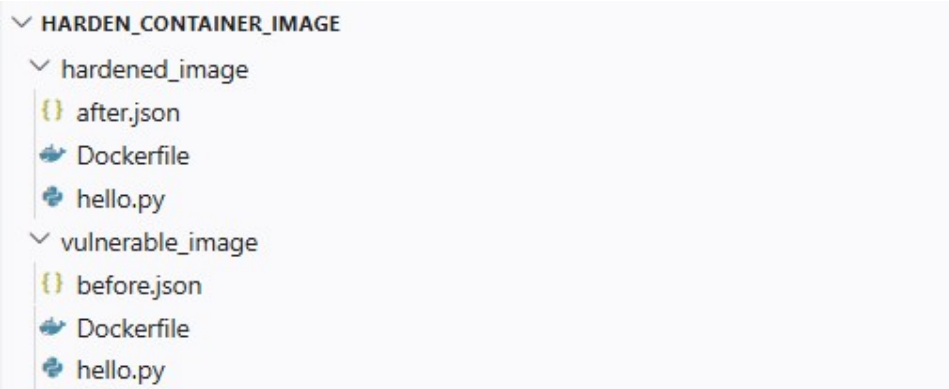
Introduction:

This practical project demonstrates how to harden a container image, starting from a deliberately vulnerable base image and then applying hardening steps.

Tools used:

- Docker
- Trivy for scanning
- Visual Studio Code

Project Structure:



Vulnerable image

App code:

```
1  from flask import Flask
2  app = Flask(__name__)
3
4  @app.route("/")
5  def hello():
6      return "Vulnerable image"
```

Dockerfile:

```
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y python3 python3-pip
RUN pip install flask==3.0.* --no-cache-dir

COPY hello.py /

ENV FLASK_APP=hello
EXPOSE 8000
USER root

CMD ["flask", "run", "--host", "0.0.0.0", "--port", "8000"]
```

This image is vulnerable because:
- it uses ubuntu:22.04 as the base image, which has many packages
- it installs python and pip
- it runs as “root”
- it uses writable filesystem by default

Building the image

docker build -t vulnerable_image:1 ./vulnerable_image

Running the image:
docker run --rm --tmpfs /tmp -p 8000:8000 vulnerable_image:1

Hardened image

Dockerfile

```
FROM alpine:3.19

WORKDIR /app

RUN apk add --no-cache python3 py3-pip &&\
    pip install --break-system-packages flask==3.0.*

COPY hello.py .

RUN addgroup -S app &&\
    adduser -S appuser -G app

ENV FLASK_APP=hello
EXPOSE 8000
USER appuser

CMD ["flask", "run", "--host", "0.0.0.0", "--port", "8000"]
```

Starting from a minimal base image the following hardening steps are applied:

- minimal layers - single RUN instruction
- non-root user - limits privileges
- no package cache.
-

Building the image

docker build -t hardened_image:1 ./hardened_image

Running the image - read-only filesystem

docker run --rm --read-only --tmpfs /tmp -p 8000:8000 hardened_image:1

Vulnerability scanning using Trivy

docker run --rm \
-v /var/run/docker.sock:/var/run/docker.sock \
aquasec/trivy image -f json vulnerable_image:1 > reports/before.json

docker run --rm \
-v /var/run/docker.sock:/var/run/docker.sock \
aquasec/trivy image -f json hardened_image:1 > reports/after.json

Before and after comparison of the CVEs reports:

Severity	Vulnerable image	Hardened image
Critical	0	0
High	5	2
Medium	2005	5
Low	126	4
Unknown	0	0
Total	2135	11

The vulnerability dropped after applying hardening steps.

References:

https://docs.docker.com/
https://trivy.dev/docs/latest/guide/target/container_image/
https://hub.docker.com/_/alpine
https://hub.docker.com/_/ubuntu/
https://botmonster.com/self-hosting/harden-docker-images-container-security-checklist/